

Agentic AI System Design Report

Bug Report Triage Agent: A Multi-Agent Workflow for Eclipse Bug Reports

Belal | Udacity Master's in AI, Capstone Project | October 2026

Overview

The task

Large open-source projects such as Eclipse receive many bug reports. Someone has to read each new report and decide what to do with it: is it a real bug, has it been reported before, how serious is it, which part of the product is affected, and does a person need to look at it. This work is called triage. In this project I built a Bug Report Triage Agent that does a first pass of this work.

For each new report, the system returns one of four suggestions: **rejected** (the text is a question or is not about Eclipse), **duplicate** (it describes a bug already known), **auto_triaged** (it proposes a severity and a component), or **needs_human_review** (the system is not confident, or something failed). The system only suggests. It never edits or closes a bug.

GitHub repo: <https://github.com/bnwiran/design-agentic-workflows-project>

Approach

The system is a multi-agent workflow built with LangGraph. Three agents have separate roles: a Router, a Duplicate-finder and a Triage agent. All three use Claude Haiku 4.5 with temperature=0 and structured output, and the Duplicate-finder also uses a TF-IDF search tool over an index of 500 known bugs. The agents are connected in a fixed order with conditional edges, so every report follows a path that can be traced and tested. LangChain (n.d.) describes this difference: a workflow follows predetermined code paths, while an agent defines its own process and tool use. I chose the workflow form because each decision can be tested on its own and a failure is easy to locate.

I chose a multi-agent design because the three decisions are different in nature: deciding the type of text, comparing a report with known bugs, and assigning labels. I tested each agent alone before connecting them. This helped later: when the Triage agent failed on one component, I fixed it by changing one line in the Triage prompt, and the other agents were not affected.

Data

The data comes from Eclipse bug data that I prepared in a separate notebook (data_prep.ipynb). It has 9,776 rows. I kept the bug id, product, title, description, component and severity, and I removed the names of assignees and reporters for privacy. Severity was mapped to three levels: low (trivial, minor), normal, and high (major, critical, blocker). After filtering, about 77% of the bugs are normal. I kept the five most common components (UI, Core, SWT, TPTP and Releng; 2,559 bugs), and the agent can also answer Other. With a fixed random seed, I split the data into a search index of 500 bugs and a test set of 21 reports. Table 1 shows the test set.

Table 1. Test set of 21 reports

Case type	Count	How it was made	Expected outcome
Sampled bugs	15	5 low, 5 normal and 5 high severity, sampled from the data; no overlap with the index	auto_triaged
Duplicates	3	Hand-written rewordings of 3 bugs in the index	duplicate
Vague	1	“Eclipse does not work”	needs_human_review
Question	1	A how-to question about the editor font size	rejected
Out of scope	1	A report about a mobile banking app	rejected

The expected outcomes were written before I ran the agent. I drafted the wording of the three edge cases (vague, question, out of scope) with Claude and reviewed it myself.

System Architecture and Design

Components

Wang et al. (2024) review how LLM-based agents are built and propose a unified framework for their architecture. I used this idea as a checklist for my design: each agent needs a role, a way to keep information, and a way to act or use tools. Table 2 shows where each component is in my system, and Figure 1 shows how they connect.

Table 2. Components of the system

Component	How it is implemented
Persona (role)	Each agent has its own system prompt: Router, Duplicate-finder and Triage. The prompts contain general rules only; no test report is written in them.
Reasoning flow	A fixed LangGraph graph: router, duplicate, triage, decide, with a human_review node. Conditional edges choose the next step. There is no open loop; the flow is bounded by a step limit.
Memory and state	TriageState holds the data of one report as it moves through the graph. A separate session list stores the result of each processed report.
Tool	A TF-IDF search over the titles of the 500 indexed bugs. It returns the top 3 similar bugs with their ids, titles and similarity scores.
Output control	Pydantic models with fixed choices (Literal) for each agent, so an invalid answer is rejected.
Safeguards	Confidence threshold, step limit, error fallback to human review, logging to a file, and suggest-only output.

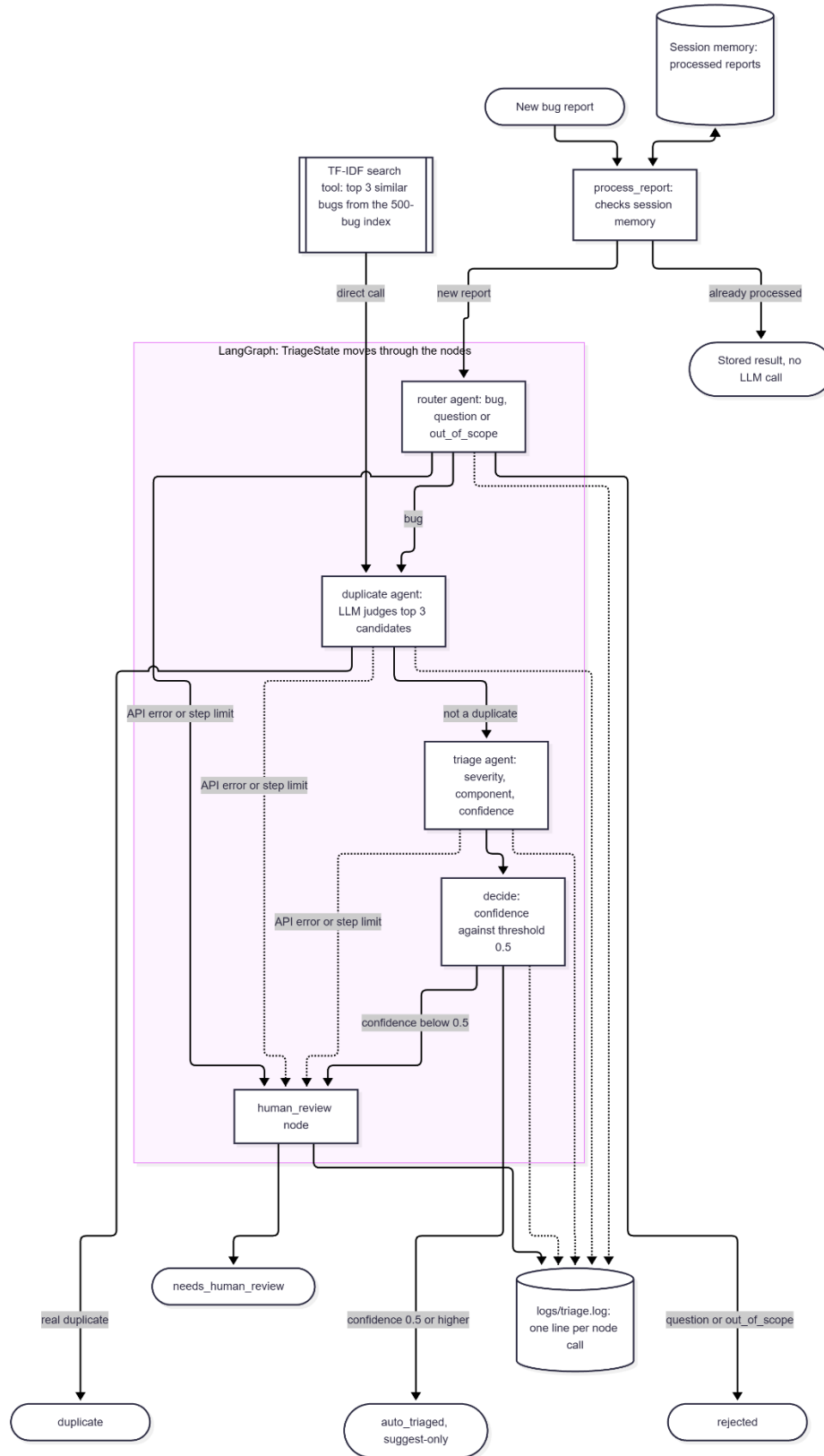


Figure 1. Architecture of the Bug Report Triage Agent, with the agents, the search tool, the state, the session memory, the safeguards and the four outputs.

Reasoning flow

A report enters through `process_report`, which first checks the session memory. A new report goes to the **Router**. If the Router says question or out of scope, the flow ends with rejected. If it says bug, the **Duplicate-finder** calls the search tool, gets the top 3 candidates, and asks the LLM to judge them. A real duplicate ends the flow with duplicate. Otherwise the **Triage** agent proposes a severity, a component and a confidence. A final decide node compares the confidence with a threshold of 0.5: at or above it, the result is `auto_triaged`; below it, the report goes to `human_review`. Any error in any node also goes to `human_review`.

Memory and tool use

The graph state is the memory of one report. It stores the report, each agent's answer, the search candidates, the step count, the final decision and a log. The session memory is a list with one record per processed report. If the same bug id arrives again, the stored result is returned and no LLM call is made. I tested this with 3 reports processed twice: the second pass came from memory, and the log file did not get new lines. The session list is only a record and a repeat guard. It does not feed the search, which uses only the 500-bug index.

The search tool is a LangChain tool, but the Duplicate-finder calls it directly in code. The search must run for every report, so letting the LLM decide whether to call it would add risk and no value. I searched the title only, because many long descriptions in the data are automatically generated and look alike.

Design choices and tradeoffs

- **Fixed graph instead of an LLM supervisor.** The flow is predictable and easy to test against expected outcomes. The tradeoff is less flexibility. This is a bounded decision flow, not an open reasoning loop.
- **Search tool called directly.** It always runs and cannot be skipped. The tradeoff is that the agent does not choose its tools.
- **One model for all agents, with temperature 0.** This is simple and cheap. The tradeoff is that all agents share the same weaknesses, and answers are not fully repeatable (see Observed Behavior).
- **Structured output with fixed choices.** Invalid values are rejected. The tradeoff is that a malformed answer sends the report to a human.
- **TF-IDF on titles.** It is simple and easy to explain, but it matches words, not meaning.

- **Confidence from the LLM only.** I first planned to combine the LLM confidence with the top search score. I dropped this because the vague report got a search score of 0.560 against an unrelated bug, so the score is not a reliable sign of a good match (see Decision Logic).

The first plan used a different model through the course API, but the key expired, so I changed only the model setup. The graph, the schemas and the tool stayed the same.

Decision Logic and Behavior

Each agent makes one decision and returns a structured answer with a short reason.

Router. It chooses bug, question or out_of_scope. The prompt tells it to choose bug even when details are few, because missing details are handled later by the confidence check. I wrote this rule with the vague test case in mind. It makes sure that a vague report reaches the Triage agent and the confidence check, instead of being rejected early.

Duplicate-finder. It receives the new report and the top 3 candidates with their similarity scores. The prompt says that a duplicate must be the same underlying problem, that sharing general words such as “does not work” is not enough, that the same component is not enough, and that the score is only a hint. The LLM must return one of the three candidate ids or an empty value. If it returns an unknown id, the code raises an error and the report goes to human review. This protects against invented ids.

Triage. It assigns a severity (low, normal, high), a component (UI, Core, SWT, TPTP, Releng or Other) and a confidence from 0 to 1. The prompt gives general definitions of each severity and component, and tells the model to give a low confidence when the report is vague, has few details, or fits several components.

Decide. This is plain code, not an LLM. It compares the Triage confidence with the threshold of 0.5. I fixed this value before running the test, and I did not tune it on the test reports.

Table 3 shows how different inputs move through the graph in the final evaluation run.

Table 3. Paths through the graph (final evaluation run)

Input	Path	Steps	Outcome
Question or out-of-scope report	Router, then end	1	rejected
Reworded duplicate (3 reports)	Router, Duplicate-finder, then end	2	duplicate
Sampled bug (15 reports)	Router, Duplicate-finder, Triage, decide	3	auto_triaged
Vague report	Router, Duplicate-finder, Triage, decide, human_review	3 agent steps	needs_human_review

The vague report shows why the LLM judgment is needed. The search tool matched “Eclipse does not work” to a bug about one specific method with a score of 0.560, which is higher than the scores of the 15 sampled reports (0.229 to 0.485). The match was based only on the words “does not work”. The Duplicate-finder rejected it, and the Triage agent gave a confidence of 0.3. In its reason, the model said that the report gives no detail about what does not work, so it could not choose a component and the report needs clarification. A decision based on the similarity score alone would have been wrong here.

Safety, Reliability, and Transparency

The system has several safeguards. Table 4 lists each one, its purpose, and what I observed in my runs.

Table 4. Safeguards

Safeguard	Purpose	Observed in my runs
Suggest-only output	A wrong result cannot damage the bug database; a person decides	Always on; no function edits a bug
Confidence threshold (0.5)	Send uncertain reports to a person	Triggered once, for the vague report (confidence 0.3); did not catch any wrong label
Structured output with fixed choices	Reject invalid severity, component or route values	Valid in all runs; invalid values also fail in a direct test
Candidate id check	Stop the LLM from inventing a duplicate id	Did not trigger
Error fallback to human review	A failed API call or bad output must not crash the flow or pass silently	Not triggered; I did not inject failures

Safeguard	Purpose	Observed in my runs
Step limit (4)	Guard against loops in future versions	Not triggered; the longest path uses 3 agent steps
Log file	Make every decision traceable	73 lines for 21 reports; one line for each node call

For transparency, every node writes one line to logs/triage.log with the bug id, node name, step count and the model’s reason. The same entries are kept in the state. These reasons are the model’s own explanation. I use them to follow the flow and to review decisions, not as proof of how the model actually decided.

The confidence threshold is the main control over automation, but it has a known weakness. Xiong et al. (2024) report that LLMs tend to be overconfident when they state their confidence in words, and that, even with larger models, the quality of this signal is still far from ideal. My results show the same problem (see Observed Behavior). For this reason, the threshold in this system is a simple guard and not a guarantee.

Observed Behavior and Limitations

All numbers in this section come from my own runs in the notebook. The values come from the final saved execution unless stated otherwise.

Overall results

All 21 reports ended with the expected decision type. The three reworded duplicates were matched to their original bugs. The search tool also ranked the original first in each case (scores 0.601, 0.646 and 0.728, against 0.215 to 0.337 for the second candidate). The vague report went to human review, and the question and the out-of-scope report were rejected.

The decision type only shows that the flow works. It does not show that the labels are right. For the 15 sampled reports, severity was correct 9 times (60%) and component 13 times. On this balanced test set, always answering normal would give 5 of 15 (33%). In the full data, normal is about 77%, so the 60% result should not be compared with that number. Severity was correct for 5 of 5 low reports, but only 2 of 5 normal and 2 of 5 high reports. The two component errors were a swap between Core and UI.

Failure case: the TPTP component

In the first run, the Triage agent got 0 of 5 TPTP reports right. It never predicted TPTP and chose UI, Releng or Other. My first TPTP description in the prompt was only “testing and profiling tools”. I looked at the 55 TPTP bugs in the search index (not at the test reports). Many of them were about profiling, tracing and test results, but also about documentation, installation guides and copyright. I rewrote only the TPTP line of the prompt to name these topics and to say that a report naming TPTP should be labeled TPTP. Table 5 shows the result.

Table 5. Before and after the prompt change (16 reports: 15 sampled and the vague one)

Run	Prompt	Product line sent	Component correct	TPTP correct	Severity correct
1	Original	Yes	8 / 15	0 / 5	9 / 15
2	New TPTP line	Yes	12 / 15	4 / 5	9 / 15
3	New TPTP line	No	14 / 15	5 / 5	9 / 15
Final evaluation	New TPTP line	No	13 / 15	5 / 5	9 / 15

Only the TPTP line changed between run 1 and run 2, and the other four components kept their scores. Severity did not change because the fix was not aimed at it. I kept all runs in the notebook, with their outputs, so the before and after can be checked.

While doing this, I found a data shortcut. All 55 TPTP bugs in the index, and all 5 TPTP test reports, have the product name Z_ARCHIVED, and no other component has it. The model could have used this as a hint. In run 3, I removed the product line from the input, and TPTP was 5 of 5, so the gain comes from the new prompt text. The final Triage agent does not receive the product name, so it does not depend on this artifact of the dataset.

The confidence signal

Six of the 15 sampled reports had a wrong label (a wrong severity, a wrong component, or both). All six were auto-triaged, because every sampled report had a confidence of 0.70 or higher. The wrong labels had confidences from 0.70 to 0.85. Table 6 shows what other thresholds would have done. This is an analysis after the test and the system did not use it.

Table 6. Threshold analysis on the 15 sampled reports (after the test, not used by the system)

Threshold	Reports sent to review	Wrong reports caught
0.50 (used by the system)	0 / 15	0 / 6
0.70	0 / 15	0 / 6
0.80	3 / 15	3 / 6
0.85	3 / 15	3 / 6

A threshold of 0.8 would catch 3 of the 6 wrong reports. The other three wrong reports have a confidence of 0.85, which is the same as six correct reports, so the model’s confidence cannot separate them. I did not change the threshold after seeing this, because that would be tuning on the test reports, and 15 reports are too few to choose a value.

Repeatability

With temperature=0, I expected identical answers, but this was not always the case. In the same execution, the runner for run 3 and the final evaluation used the same settings. They gave 14 of 15 and 13 of 15 correct components. One report changed from Core (confidence 0.6) to UI (confidence 0.7). In an earlier execution, the same report also showed different confidence values. This means that small differences between executions are normal for this system, and my numbers describe the saved execution, not a fixed property.

Limitations

- **Small and easy test set.** There are 21 reports, only 5 per severity level, and the three duplicates are rewordings that share many words with the originals. The three edge cases are also easy. The results are not a general measure of performance.
- **One execution per setting.** I did not repeat runs enough to measure variation, and the repeatability check above shows that variation exists.
- **The failure case was found on the test reports.** The new prompt text came from the index, but the test reports showed me where to look. So run 2 is not a clean test on unseen data.
- **The index contains only fixed bugs, and the search matches words, not meaning.** A real duplicate with different words may be missed.
- **Severity is weak.** The agent often confuses normal with low or high.

- **The reasons in the log are model-written.** They help review, but they do not prove how a decision was made.

Ethical and Responsible Use Considerations

The main ethical concern for this system is **accountability and over-trust in automatic decisions**. The system labels a report as `auto_triaged` when its own confidence is high, but my results show that high confidence does not mean a correct label. All six wrong reports were auto-triaged. Three of the five high-severity reports received a lower severity (two normal and one low). In a real project, a serious bug labeled as low or normal could wait longer than it should. If people start to trust the label because the system sounds confident, this error could pass without anyone noticing. The NIST AI Risk Management Framework treats human-AI interaction as part of managing AI risk (National Institute of Standards and Technology, 2023).

The design reduces this risk in three ways. The system is suggest-only, so a person owns the final decision. The log keeps a trace of every decision, so an error can be reviewed. And this report states the limits of the confidence value openly. These measures help but do not remove the risk. A better design would, for example, always send reports with a predicted low or normal severity to a person, or show the confidence together with a warning that it is self-reported.

A second concern is **bias in the data and labels**. About 77% of the bugs in the data are normal, and the severity labels come from people's past decisions, which may be inconsistent. A system trained or tuned on such labels can repeat their errors. I also found a dataset artifact (`Z_ARCHIVED`) that can work as a shortcut for one component. I removed it from the input, but there may be other shortcuts that I did not find.

A third concern is **privacy**. Bug reports and logs can contain personal or company information. I removed the names of assignees and reporters from the data, and I send only the title and the first 1,500 characters of the description to the model. However, the text still goes to an external API. A real deployment would need a filter for personal data and a review of the provider's data terms.

Future Improvements

- **Better confidence.** Xiong et al. (2024) found that asking a model several times and measuring how much the answers agree improves the detection of wrong answers. I could add this check and compare it with the current confidence, using a separate validation set.
- **Safer routing of severity.** Send predicted high severity, and possibly low and normal, to a person until the severity results improve.
- **A larger and harder test set.** Use real duplicates from the bug tracker, more reports per severity level, and a separate validation set for prompt changes, so the test reports stay unseen.
- **Repeated runs.** Run each setting several times and report the range, to measure the variation I saw.
- **Better search.** Compare TF-IDF with embedding-based search, and add processed reports to the index so the session memory also helps duplicate detection.
- **Test the failure paths.** Inject API errors and invalid answers to show that the error fallback and the step limit work, because they did not trigger in my runs.
- **Privacy filter.** Remove personal data from the text before it is sent to the model.

Acknowledgment of AI Assistance

I used Claude (Anthropic) as a colleague for planning, for reviewing my results, and for drafting parts of the code, the prompts and this report. I ran every cell, checked the outputs, made the design decisions, and revised the text so that it matches my notebook. The wording of the three edge-case test reports was drafted with Claude and reviewed by me.

References

- LangChain. (n.d.). *Workflows and agents*. LangGraph documentation. Retrieved October 4, 2026, from <https://docs.langchain.com/oss/python/langgraph/workflows-agents>
- National Institute of Standards and Technology. (2023). *Artificial intelligence risk management framework (AI RMF 1.0)* (NIST AI 100-1). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.AI.100-1>
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J.-R. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), Article 186345. <https://doi.org/10.1007/s11704-024-40231-1>
- Xiong, M., Hu, Z., Lu, X., Li, Y., Fu, J., He, J., & Hooi, B. (2024). Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. In *The Twelfth International Conference on Learning Representations*. <https://arxiv.org/abs/2306.13063>